Unidad 5. HTML5, CSS, JavaScript y depuración de errores



Contenido

HTML 4
Estructura de un HTML 4
La cabecera <head> 4
El cuerpo <body> 6
Explicación del ejemplo 7
Etiquetas de HTML 9
Párrafos y encabezados 9
Etiquetas para crear listas 10
Atributos de las etiquetas 10
Imágenes 11
La indentación 12
Comentarios 12
Estilos 13
Tamaño de fuente 13
Color de fuente 13
Familia de fuentes tipográficas 13
Color de fondo 14
Etiqueta iframe 15
CSS 16
Incluir CSS en HTML 16
La sintaxis de CSS 17
Comentarios 19
Todos los elementos de HTML son posibles selectores 19
Las clases 20
Los identificadores 21
Introducción a JavaScript. ¿Qué es JavaScript? 23
Valores 23
Tipos de JavaScript (valores) 23
Objetos 24
Operadores y expresiones 24

Programa, sentencias y comentarios 25
Variables 25
Sintaxis 27
Delimitación de sentencias 27
Nombre de variables 27
Orden en la creación de variables 27
Declaraciones y expresiones 29
Funciones útiles 29
Script con fecha 29
Script con expresión 30
Funciones 30
Condicionales y bucles 31
Depuración de JavaScript 34
Corrección de errores con Firefox 34
Depuración de errores con Chrome 40

HTML

Es un lenguaje de marcas para formatear y estructurar un documento, que puede leerse en cualquier navegador.

```
<html> … …</html>
```

Estructura de un HTML

Un documento HTML tiene tres etiquetas que describen la estructura general de un documento y dan una información sencilla sobre él:

- <html>
- <head> y
- <body>

Dentro de un HTML debemos escribir siempre

<!doctype html>

en la primera línea. Esto le dice al navegador cuál es el lenguaje que está leyendo (en este caso, HTML5).

Doctype comienza a utilizarse en HTML4.01 y XHTML https://validator.w3.org/

La cabecera <head>

El contenido que irá dentro de nuestro archivo HTML irá en medio de las etiquetas de inicio <html> y de cierre </html>.

El archivo siempre consiste de dos partes: la cabecera y el cuerpo. Vamos a centrarnos en la cabecera.

Tiene una etiqueta de inicio y una de cierre.

La cabecera incluye información importante, como el título de la página web. El título **<TITLE>** es el nombre que aparece en la cabecera del visualizador y en los buscadores, de ahí la importancia de que sea significativo.



La etiqueta meta define varios tipos de metadatos.

El atributo charset especifica la codificación usada en nuestra página.

```
<meta charset="character_set">
```

Charset utf-8 es la **U**nicode **T**ransformation **F**ormat **8**-bit representa el código de caracteres UNICODE. Es compatible con ASCII.

Permite visualizar los caracteres de todos los idiomas.

Hay otras etiquetas como ***description*** o ***keywords***. Estas etiquetas meta se utilizan para dar información sobre el contenido de la página web pero normalmente no son mostradas en pantalla. Es información relevante para el navegador porque permite realizar las búsquedas y conocer los datos más relevantes de la página pero como hemos dicho no son visibles para el usuario.

Un elemento importante a incluir en la cabecera del documento es **<link>.**

Esta etiqueta es la que se utiliza para incorporar a nuestra página otros archivos externos con imágenes, iconos, código,... Con **<link>** necesitaremos utilizar dos atributos: **rel** y ***href***.

A continuación se muestra un ejemplo típico de una cabecera:

```
<!DOCTYPE html>
<html>
	<head>
	<meta charset="UTF-8">
	<meta name="description" content="Mi primer HTML">
	<meta name="keywords" content="HTML , MappinGIS, Leaflet">
	<link rel="stylesheet" type="text/css" href="mihojadeestilos.css">
	<title> MappingGIS.com </title>
	</head>
</html>
```

El atributo **rel** indica la relación entre el documento y el archivo que estamos cargando. En este ejemplo el atributo **rel** tiene el valor stylesheet (hoja de estilos) que está indicando que el archivo que incorporamos ("mihojadeestilos.css") es un archivo CSS. Durante este curso veremos algún ejemplo de archivo css y como trabajar con él. El atributo **href** indica la ruta donde se encuentra el archivo que vamos a utilizar.

El cuerpo <body>

El contenido en el cuerpo es lo que será visible en la página web. Contiene los elementos visibles en la página web: párrafos, imágenes, encabezados, listas, etc.

```
<!DOCTYPE html>
<html>
 <head>
      <title>MappingGIS.com</title>
 </head>
        <body>Formación que impulsa tu perfil GIS</body>
</html>
```

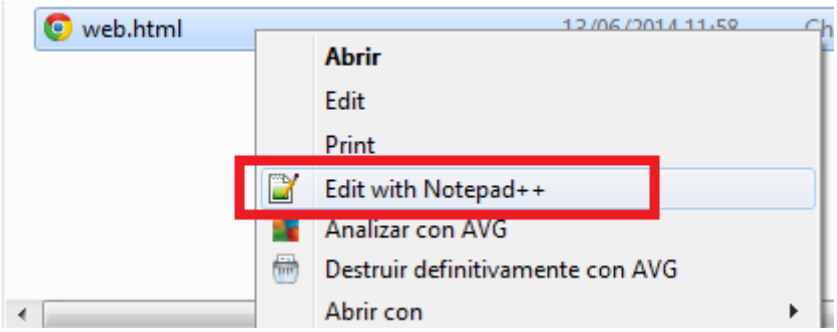
Edita el archivo web.html que creamos en la unidad 1 y que hemos guardado en el directorio \data_dir\www

Si haces doble clic sobre él, se abrirá como una página web en tu navegador, pero si lo abres con un editor de texto, como Notepad++ o Sublime Text nos aparecerá su código fuente.

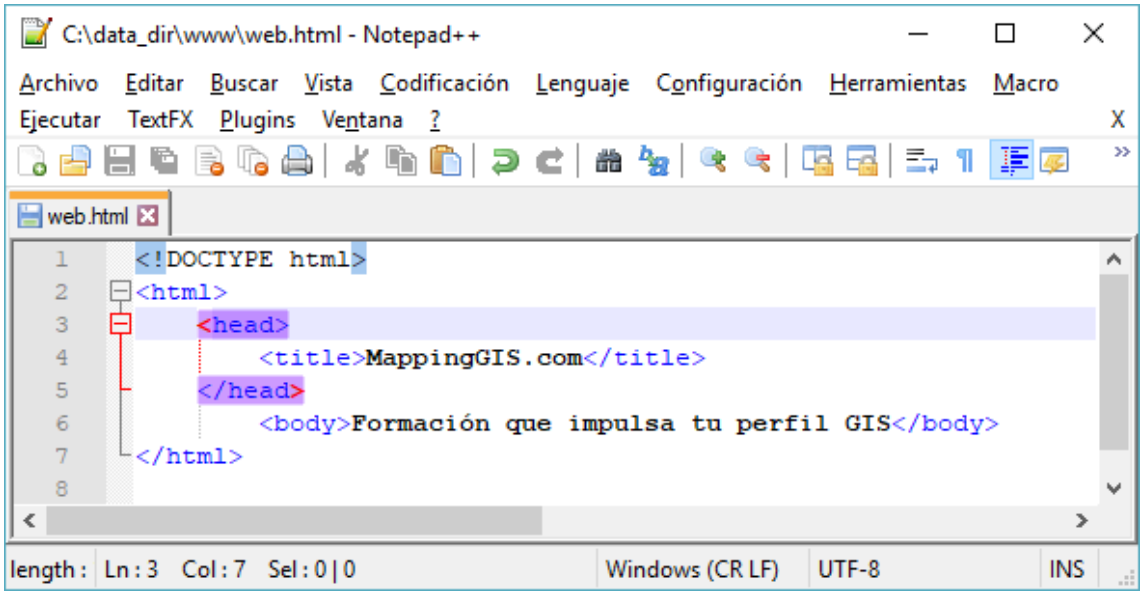
Antes de continuar instala uno de los editores anteriores.

La instalación de Notepad++ es muy sencilla. Accede a la página de descargas y haz clic sobre *Download*. Ejecuta el archivo .exe que se descarga y sigue los pasos.

Una vez que se haya instalado haz clic con el botón derecho del ratón y clic sobre *Edit with Notepadd++.*



Copia el texto anterior y pégalo en el editor:



Explicación del ejemplo

<!doctype html> especifica que el lenguaje es HTML5.

<html> Indica al navegador que abre un archivo HTML.

<head> Apertura de la cabecera del documento. Sección que no se muestra en la pantalla.

<title> MappingGIS.com **</title>** Título de la página.

</head> Cierre de la cabecera del documento.

<body> Apertura del cuerpo del documento. Esta sección aparece en la pantalla. **Formación GIS y difusión tecnológica**. Texto que va a ser mostrado por el navegador.

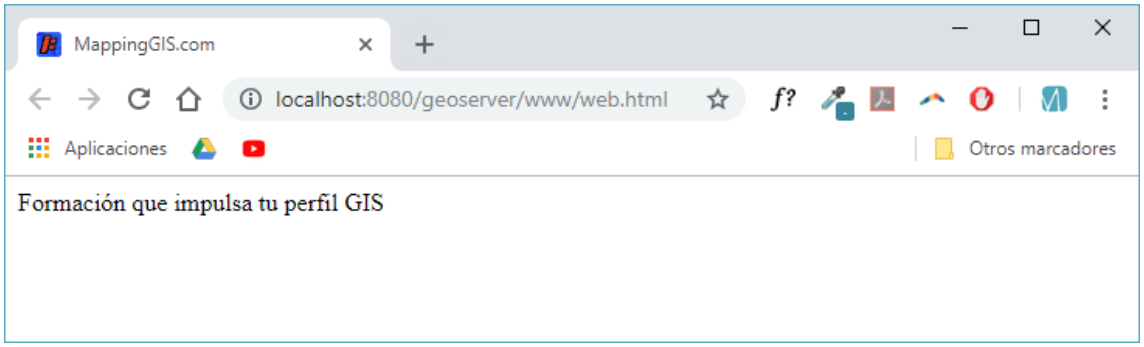
</body> Cierre del cuerpo del documento.

</html> Cierre del archivo HTML

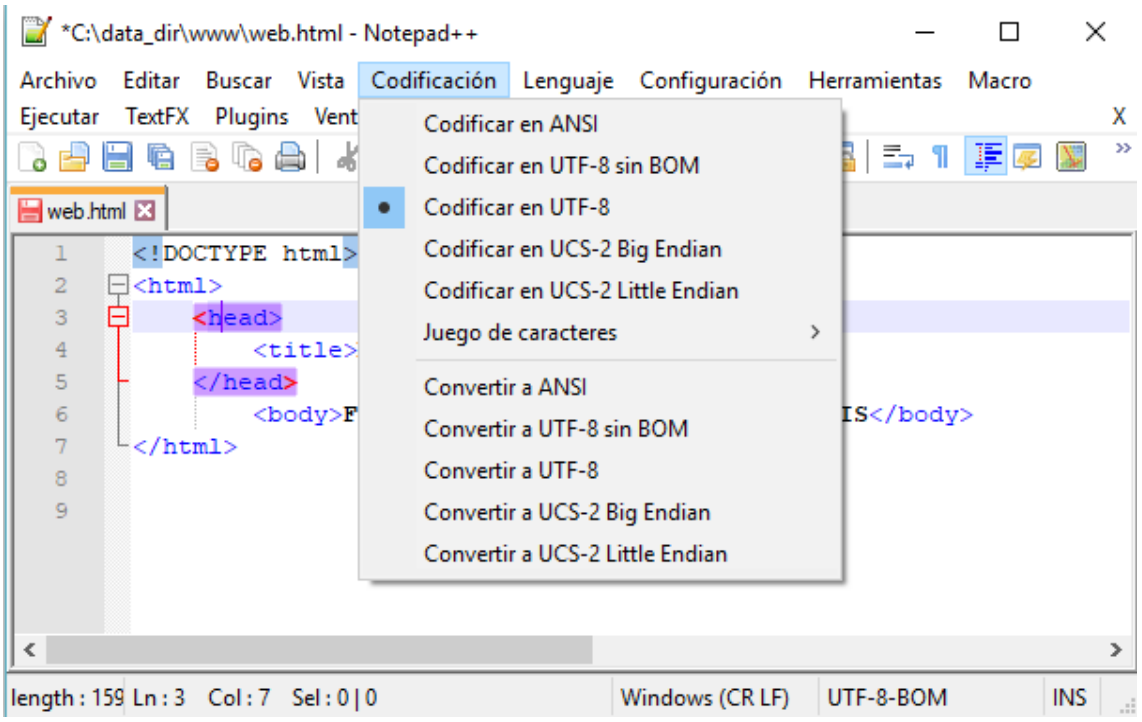
Fíjate que al situar el cursor sobre la etiqueta <head> también se ilumina la etiqueta de cierre </head>

Abre la web en tu navegador, ve a: http://localhost:8080/geoserver/www/web.html

En la pestaña verás el título de la página web y en contenido una breve descripción:



Si al abrir el navegador te aparecen caracteres extraños en el texto y no reconoce las tildes, ñ, etc hay un problema con la codificación del archivo. Con un editor de texto, como Notepad++ o Sublime Text, puedes modificar su codificación a UTF-8:



Ejercicio nº1 Modifica tanto el título de la página como el texto del cuerpo y realiza una captura de pantalla con el resultado.

Video: Visualiza el video correspondiente a esta sección

Etiquetas de HTML

1. Lo que va dentro de los signos "<" y ">" se denominan **etiquetas**.

2. Las etiquetas casi siempre vienen en pares: una etiqueta de inicio y una etiqueta de cierre.

3. Un ejemplo de una etiqueta de inicio: <html>

4. Un ejemplo de una etiqueta de cierre: </html>

5. El texto que se encuentra entre dos etiquetas es el **contenido** del elemento.

Imagina que las etiquetas son como paréntesis; cuando abres uno, debes cerrarlo. Las etiquetas también **pueden anidarse**, así que debes cerrarlas en el orden correcto: la etiqueta abierta más recientemente debe ser la primera que se cierre, como en el ejemplo de abajo.

```
<primera etiqueta>
        <segunda etiqueta>¡Algún texto!</segunda etiqueta>
</primera etiqueta>
```

En el ejemplo:

```
<b>Este texto es en negrita.</b>
```

El elemento HTML comienza con la **etiqueta de apertura ** (etiqueta que nos muestra el texto en negrita).

El **contenido** de dicho elemento es: Este texto es en negrita.

El fin del elemento HTML es con la **etiqueta de cierre **.

Párrafos y encabezados

Cada párrafo requiere etiquetas de inicio y de cierre: y . Colocamos el contenido en medio de las etiquetas.

El siguiente paso es colocarle encabezados a nuestros párrafos, usando las **etiquetas de encabezado**.

HTML nos permite tener más de un tamaño de encabezado. Hay seis tamaños de encabezado: **<h1>** es el más importante y **<h6>** el menos importante.

Etiquetas para crear listas

Para crear una lista en HTML utilizaremos el elemento <ul> en combinación con <li>, como en el ejemplo siguiente:

```
<!DOCTYPE html>
<html>
<head>
	<meta charset="UTF-8">
	<meta name="description" content="Mi segundo HTML">
	<meta name="keywords" content="HTML , MappinGIS, Leaflet">

	<title> Mis bebidas preferidas </title>
</head>
<body>
	<h2>Lista de capas base</h2>
	<ul>
		<li>Stamen</li>
		<li>OpenStreetMap</li>
		<li>Google Maps</li>
	</ul>
</body>
</html>
```

La etiqueta <ul> del ejemplo anterior crea una lista de elementos sin un orden definido, (se mostrarán en el mismo orden que han sido escritos).

Para declarar cada elemento utilizaremos <li>.

Se ha añadido además una etiqueta de encabezado <h2> que mostrará el texto “Lista de capas base” al ejecutar la página en el navegador.

Atributos de las etiquetas

Muchas etiquetas llevan atributos. Los atributos proveen de mayor información a los elementos HTML.

- Los atributos siempre van con la estructura: **nombre="valor".**
- Los atributos siempre van en la **etiqueta de apertura**.
- Los valores siempre hay que ponerlos **entre comillas**.

Un ejemplo de los atributos sería:

```
<body bgcolor="#FFFF00">
Hola a todos. <b>Este texto es en negrita</b>
</body>
```

Resultado

Hola a todos. **Este texto es en negrita**

En la etiqueta <body> podemos observar el atributo bgcolor= (color de fondo) y el valor "#FFFF00"(representa el color amarillo en hexadecimal).

Esto quiere decir que el color de fondo de la página será amarillo.

***Nota**: La etiqueta y su correspondiente par de cierre indican que el texto incluido entre ambas se mostrará en negrita.*

Otro ejemplo del uso de atributos los podemos aplicar al caso de las listas que explicamos en el ejemplo anterior.

<ul style="list-style-type:square">

Mediante el atributo style podemos indicar el aspecto que tendrá el marcador de cada item de la lista, por ejemplo en este caso le estamos indicando que sea un cuadrado.

Ejercicio nº2 Escribe un código HTML para crear una lista con tus libros, películas de cine,... preferidas. Haz que el marcador del item de la lista sea un cuadrado y que el primer elemento de la lista esté en negrita. Elige además un color para el fondo del cuerpo del documento.

Haz una captura de pantalla con el resultado y pégalo en la hoja de resultados

Imágenes

Para añadir imágenes a un sitio web solamente necesitas una etiqueta de imagen: <img>. Esta etiqueta es un poco diferente a las demás. En vez de colocar el contenido en medio de las etiquetas, le dices a la etiquetas dónde está la imagen, usando src.

La dirección web (o **URL**, por **l**ocalizador de **r**ecursos **u**niforme) después de src= le dice a la etiqueta <img> de dónde obtener la imagen.

```
<!DOCTYPE html>
<html>
        <head>
                <title> Curso online desarrollo de aplicaciones web
mapping</title>
        </head>
        <body>
```

```
                <p>Insertando una imagen</p>
                <img src="https://mappinggis.com/wp-
content/uploads/2014/02/resultado.png">

        </body>
</html>
```

La indentación

La **indentación** es el espaciado que hay desde el margen en cada línea. Podrás ver que cuando ponemos etiquetas dentro de otras etiquetas, las indentamos aún más.

JavaScript no es un lenguaje indentado (es recomendable pero no necesario),

La indentación hace que tu código se pueda leer mejor.

Trata de seguir este estilo de indentación cuando estés escribiendo HTML, para que no te confundas:

```
<!DOCTYPE html>
<html>
        <head>
                <title></title>
        </head>
        <body>
        </body>
</html>
```

Comentarios

Puedes incluir pequeñas notas, que el navegador no mostrará, en tu código HTML. Los comentarios son muy útiles en nuestro código para ayudarnos a recordar porqué hicimos ciertas cosas.

Los comentarios en HTML comienzan con <!-- y terminan con -->, así:

```
<!-- ¡Este es un ejemplo de un comentario en HTML! -->
```

Ejercicio nº3. Sustituye el párrafo *Insertando una imagen* por el mismo texto como un comentario.

Haz una captura de pantalla del código utilizado.

Estilos

Tamaño de fuente

Como recordarás, HTML es el esqueleto de la página web, y CSS te permite darle al esqueleto algo de piel y de maquillaje.

Pero es posible hacer algo de **CSS en líneas**. Esto simplemente quiere decir que podemos poner algo de estilo en nuestro HTML sin preocuparnos por un archivo CSS aparte.

Recuerda que y son **etiquetas** de inicio y de cierre.

Le podemos dar a las etiquetas más instrucciones mediante la inclusión de **atributos** en la etiqueta de inicio. Un atributo es solamente una característica o una descripción del contenido en el elemento.

Vamos a cambiar el tamaño del texto usando el atributo **style**. Escribimos igual a font-size, seguido de dos puntos, el tamaño que deseamos, y finalizamos con px (abreviatura de "pixeles"). Por ejemplo:

```
<p style = "font-size: 12px"></p>
```

Color de fuente

Para cambiar el color del texto, simplemente añade el atributo style en la etiqueta de inicio, luego haz que sea igual a "color:blue" (color azul) o cualquier otro color. Por ejemplo:

```
<p style="color:red"></p>
```

Si quieres cambiar el color *y* el tamaño del texto basta con añadir un punto y coma en medio de cada parte. Por ejemplo:

```
<p style="color: red; font-size:8px"></p>
```

Familia de fuentes tipográficas

Para cambiar el tipo de fuente, añade el atributo style en la etiqueta de inicio, luego hazlo igual a "font-family:Helvetica" (o cualquier fuente que quieras). Por ejemplo:

```
<p style="font-family:Futura"></p>
```

Ejercicio nº 4. Añade un párrafo que lleve como nombre **Mapa de Leaflet** pero de color azul, tamaño 18 y el tipo de fuente Tahoma.

Haz una captura de pantalla y pégala en la hoja de resultados.

Color de fondo

De nuevo vamos a usar el atributo **style**. Lo haremos igual a "background-color: red" (color de fondo: rojo) o cualquier color que quieras. ¡Es así de fácil!

```
<body style="background-color: blue">
```

Etiqueta iframe

Es posible incrustar mapas web en la página haciendo uso de la etiqueta <iframe>.

La etiqueta <iframe> sirve para crear un espacio dentro de la página donde se puede incrustar otra web.

Es un cuadrado cuyas dimensiones debe especificar el desarrollador en la propia etiqueta iframe, que tiene asociada una página web que se carga en dicho espacio. Esa página web tendrá sus propios contenidos y estilos, independientes del contexto donde se está mostrando. Además será perfectamente funcional: si tiene enlaces se mostrarán en ese mismo espacio y si tiene scripts o aplicaciones dentro se ejecutarán también de manera autónoma en el espacio reservado al iframe.

```
<iframe style="border: none;" height="400" width="600"
        src="http://localhost:8080/geoserver/www/web.html"></iframe>
```

CSS

CSS (cuyas siglas significan **C**ascading **S**tyle**S**heets, u Hojas de Estilo en Cascada) es un lenguaje usado para describir la apariencia y el formato de tu HTML.

Una **hoja de estilo** es un archivo que describe como debería verse un archivo HTML.

Decimos que estas hojas de estilo están **en cascada** porque las hojas pueden aplicar el formato cuando corresponde más de un estilo. Por ejemplo, si indicas que todos los párrafos deben tener una fuente de color azul, pero señalas que un párrafo específico debe tener fuente de color rojo, CSS lo puede hacer.

El archivo stylesheet.css contiene toda la información de estilo de CSS: en dónde deberían ir los elementos de HTML, cuál color deberían tener, qué tamaño deberían tener, y más.

Existen dos razones principales para separar tu forma/formato (CSS) de tu contenido/estructura funcional (HTML):

a. Puedes aplicarle el mismo formato a diversos elementos de HTML sin reescribir el código (*p.ej.*, style="color:red":) una y otra vez.
b. Puedes aplicarle una apariencia y un formato similares a varias páginas de HTML desde un solo archivo CSS.

Incluir CSS en HTML

Para que tu archivo HTML pueda ver la información de CSS puedes colocar una etiqueta <link> entre las etiquetas <head></head> de tu página HTML. Tu etiqueta <link> necesitará tres atributos:

1. Un atributo type que será igual a "text/css"

2. Un atributo rel que será igual a "stylesheet"

3. Un atributo href que debe apuntar a la dirección web de tu archivo CSS

```
<!DOCTYPE html>
<html>
        <head>
                <link            type="text/css"            rel="stylesheet"
href="stylesheet.css"/>
                <title>Título</title>
        </head>
        <body>
                <p>párrafo</p>
        </body>
</html>
```

Como nada va en medio de las etiquetas<link></link>, está bien usar un solo par de <> para que sirvan como etiquetas de inicio *y* de cierre.

La sintaxis de CSS

La sintaxis de CSS es distinta de la de HTML. El formato general se ve así:

```
selector {
    propiedad: valor;
}
```

Un **selector** puede ser cualquier elemento de HTML, como <p>, <img>, o <table>. Simplemente es necesario obviar los <>. Para hacer que el texto de un párrafo tenga color rojo con CSS, escribirías:

```
p {
    color: red;
}
```

Una **propiedad** es un aspecto de un selector. Por ejemplo, puedes cambiar font-family, color, y font-size del texto en tus páginas web (además de muchos más).

Un **valor** es una posible configuración de una propiedad. color puede ser rojo, azul, negro o cualquier otro color; font-family puede ser prácticamente cualquier fuente; y así sucesivamente.

Debes finalizar cada propiedad-valor con un punto y coma (;). Así es como CSS sabe que ya terminaste de trabajar en un par y que estás listo para trabajar en el siguiente.

Además un selector puede tener muchas propiedades. Por ejemplo, si quieres establecer la fuente, el color de fuente y el tamaño de fuente de un párrafo, sencillamente puedes escribir:

```
p {
    font-family: Arial;
    color: blue;
    font-size: 24px;
}
```

Ahora vamos a incluir código CSS dentro de nuestro HTML. Deberás incluir las etiquetas <style> y </style> **dentro de la etiqueta <head>.**

```
<!DOCTYPE html>
<html>
      <head>
            <style>
            <!-- aquí dentro va nuestro código CSS -->
            </style>
      </head>

      <body> </body>
</html>
```

Dentro de las etiquetas de estilo incluiremos nuestras propiedades y valores para el párrafo ***Mapa de Leaflet*.**

Elimina las definiciones de estilo que incluimos en el ejercicio anterior. La línea debería quedar así:

```
<p> Mapa de Leaflet</p>
```

Ahora incluye este código dentro de las etiquetas `style`:

```
p {
    font-family: Arial;
    color: red;
    font-size: 18px;
}
```

Ejercicio nº5. Incluye las propiedades y valores dentro de las etiquetas de estilo CSS y sube el html a la plataforma de formación.

Por supuesto podemos utilizar las propiedades de estilo para dar formato a nuestro mapa. En los ejercicios de OpenLayers y Leaflet veremos cómo hacerlo.

```
<style>
      #map {
            Width: 50 px
            height: 580px;
            }
</style>
```

Comentarios

Como hemos visto, los comentarios de HTML se ven así:

```
<!--¡Soy un comentario!-->
```

Por otro lado, los comentarios de CSS lucen así:

```
/*¡Soy un comentario!*/
```

Volvemos a insistir: escribir buenos comentarios es un buen hábito para recordar porqué hicimos algunas cosas.

Todos los elementos de HTML son posibles selectores

Podemos usar todos de elementos de nuestro HTML como selectores de CSS: le podemos dar estilo a las etiquetas de HTML <h1></h1> con el selector de CSS h1, a con p, y así sucesivamente.

Además existen dos selectores importantes que puedes usar, los elementos de HTML: de **clase** y de **identificadores**.

Video: Visualiza el video correspondiente a esta sección

Las clases

Las clases son útiles cuando tienes un montón de elementos que deberían tener el mismo estilo. En vez de aplicar las mismas reglas a varios selectores, simplemente puedes aplicar la misma clase a todos esos elementos de HTML y luego definir el estilo para esa clase en la pestaña de CSS.

Las clases son asignadas a los elementos de HTML usando la palabra class y un signo de igual, así:

```
<div class="cuadrado"></div>
<img class="cuadrado"/>
<td class="cuadrado"></td>
```

En CSS, las clases se identifican con un punto (.), así:

```
.cuadrado {
    height: 100px;
    width: 100px;
}
```

Esto te permite tomar elementos de diferentes tipos y darles el mismo estilo. Entenderemos mejor como se utiliza con un ejemplo.

```
<!DOCTYPE html>
<html>
<head>
      <meta charset="UTF-8">
      <title> Practicando con clases </title>
      <style>
            span.nota {
            font-size: 120%;
            color: red;
            }
      </style>
</head>
<body>
<h1>Es importante proteger <span class="nota">el medio ambiente</span> de
nuestro planeta</h1>

<p>También es importante conservar la <span class="nota">biodiversidad</span>
de la tierra y sus océanos, para lo cual es imprescindible reducir la emisión
a la atmósfera de <span class="nota"> gases de efecto invernadero</span>
responsables del calentamiento global. </p>

</body>
</html>
```

En el código anterior cada vez que el navegador se encuentre con la definición de clase aplicará el estilo CSS que hemos creado. Estamos utilizando también una

etiqueta **** que lo único que hace es separar o definir una zona del texto. La etiqueta span no hace nada por si sola, es la definición de la clase la que nos permite asignar estilos a esta parte del texto, lo cual conseguimos escribiendo span.nota dentro de las etiquetas <style> de la cabecera. Como explicamos antes la referencia a una clase se realiza colocando un punto delante del nombre de la clase, que en esta ocasión es .nota.

Ejercicio nº6. Utilizando como modelo el código del ejemplo anterior escribe un texto con un encabezado y varios párrafos en los que se destaquen algunas de sus partes.

Haz una captura de pantalla del resultado y pégalo en la plantilla de resultados.

Los identificadores

Por otra parte, los identificadores son la mejor opción cuando tenemos exactamente *un* elemento que debe tener cierto estilo.

Los identificadores son asignados a los elementos de HTML usando la palabra **id** y un signo de igual:

```
<div id="primero"></div>
<div id="segundo></div>
<p id="intro"></p>
```

Los identificadores se reconocen en CSS con un signo (#):

```
#primero {
    height: 50px;
}
#segundo {
    height: 100px;
}
#intro {
    color: #FF0000;
}
```

Esto te permite aplicarle estilo a una sola instancia de un selector, en vez de a *todas* las instancias. En el ejemplo siguiente veremos cómo hacerlo.

```
<!DOCTYPE html>
<html>
<head>
      <meta charset="UTF-8">
```

```
        <title> Practicando con clases </title>

<style>
        #texto1 {
                font-size: 120%;
                color: red;
        }

        #texto2 {
                font-family: Arial;
                color: blue;
                font-size: 18px;
        }

</style>

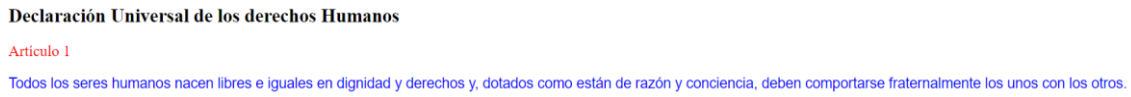
</head>

<body>

<h2>Declaración Universal de los derechos Humanos</h2>
<p id ="texto1">Artículo 1 </p>
<p id ="texto2">Todos los seres humanos nacen libres e iguales en dignidad y
derechos y, dotados como están de razón y conciencia, deben comportarse
fraternalmente los unos con los otros.</p>

</body>
</html>
```

El código anterior da como resultado lo siguiente:



Ejercicio nº7. Utilizando como modelo el código del ejemplo anterior escribe dos textos a los que se les asigne estilos diferentes mediante el empleo de dos identificadores "id".

Haz una captura de pantalla y pégala en la plantilla de resultados.

Introducción a JavaScript. ¿Qué es JavaScript?

Es uno de los lenguajes de programación más importante. ¿Por qué? Porque en él se programan las aplicaciones para internet y la Web.

Los navegadores modernos soportan la versión JavaScript 1.5 (diciembre de 2009).

Valores

JavaScript tiene todos los valores que cabría esperar de los lenguajes de programación: booleanos, números, cadenas, matrices, etc. Todos los valores en JavaScript tienen propiedades. Cada propiedad tiene una clave (o nombre) y un valor. Piensa en los campos de un registro. Se utiliza el operador punto (.) para leer una propiedad (.):

```
value.propKey
```

Un ejemplo: La cadena 'texto' tiene la propiedad length

```
> var str = 'texto';

> str.length

5
```

Tipos de JavaScript (valores)

- **number** son cantidades y puedes hacer operaciones matemáticas con ellas. `1, 2, 10.5`.
- **strings** son secuencias de caracteres, como las letras a-z, los espacios, e incluso los números. Los strings son muy útiles como etiquetas, nombres y contenido para tus programas. Para escribir un string, debes ponerlo *entre comillas*:

  `"¿Cuál es tu nombre?"`

- **booleanos.** Un booleano puede tener dos valores:
  - true (verdadero)
  - false (falso).

  Puedes usarlos en tu código haciendo sentencias que **sean igual** a `true` o `false`.

`10 > 3` es igual a `true`

`5 < 4` es igual a `false`

- **"undefined":** undefined, null

Objetos

Representan agregaciones estructuradas de valores.

Los objetos se agrupan en clases: Object, Array, Date

Todos los valores que no son primitivos son *objetos*. Los tipos más comunes son:

- Objetos planos (tipo **objet**) pueden ser creados por objetos literales:

```
{
        Nombre: 'Felipe',
        Apellidos: 'González'
}
```

El anterior objeto tiene dos propiedades: el valor de la propiedad nombre es 'Felipe', el valor de la propiedad apellido es 'Gonzalez'.

- Matrices (tipo **Array**) pueden ser creados por literales de matriz:

```
['cities', 'countries', 'ocean']
```

La matriz anterior tiene tres elementos que se puede acceder a través de los índices numéricos. Por ejemplo, el índice de 'cities' es 0.

- Las expresiones regulares (tipo RegExp) pueden ser creados por literales de expresiones regulares:

```
/ ^ a + b + $ /
```

- Objeto nulo

Operadores y expresiones

Con los tipos y objetos podemos crear expresiones utilizando operadores.

Por ejemplo las operaciones matemáticas (+, - , *, /)

```
5 + 5 => 10
```

Programa, sentencias y comentarios

Un **programa** es una secuencia de sentencias que se ejecutan en el orden en que han sido definidas.

Las **sentencias** realizan tareas al ejecutarse en un ordenador:

```
var x = 5
```

Los **comentarios** son muy útiles para explicar partes difíciles de tu código o como recordatorios para ti mismo.

JavaScript tiene dos tipos de comentarios: comentarios de una sola línea y comentarios multilínea.

Comentarios de una sola línea comienzan con **/ /** y terminan al final de la línea:

```
x + +; // comentario de una sola línea
```

Comentarios de varias líneas se delimitan con **/ *** y ***** **/**

```
/ * Este es un
comentario
de varias líneas
* /
```

Es recomendable utilizar comentarios monolínea //

Variables

Para escribir un código que sea más complejo, necesitamos una forma de 'guardar' los valores de nuestro código. Esto lo hacemos definiendo una variable con un nombre específico, que distingue entre mayúsculas y minúsculas.

En JavaScript debemos declarar las variables a través de la palabra reservada **var** seguida de la variable.

Se pueden crear varias variables en una sentencia, separando las definiciones por comas.

```
var nombre; // declaramos la variable 'nombre'

> var x; //creamos la variable x y asigna el valor undefined
> x = 1; //asignamos el valor 1 a x
> y = 2; //asignamos el valor 2 a y. Esta sentencia daría un error
porque no se ha definido la variable y

      ReferenceError: y is not defined
```

Podemos declarar una variable y asignarle un valor al mismo tiempo:

```
var osm = L.tileLayer();//crea osm y le asignamos el valor de una capa
de teselas
```

El valor de una variable se puede cambiar fácilmente mediante una sentencia de asignación.

```
var osm = L.tileLayer.wms();
```

Dentro de una variable podemos incluir cualquier cosa (un número, un string, una fecha, etc)

Sintaxis

Delimitación de sentencias

Los ";" delimitan el final de una sentencia.

Son opcionales, pero se recomienda utilizarlo siempre, porque su omisión puede dar lugar a sorpresas. Además los programas son más legibles y seguros.

```
var x = 1; //bien

var x; //no recomendable porque esta como undefined

x = 1  //no recomendable porque falta el ;

var x = 1 //no recomendable porque falta el ;
```

Nombre de variables

El nombre de una variable debe comenzar por **letra**, _ o **$**

```
x, map, map1, _map, &f
```

Por lo tanto un nombre de una variable no debe comenzar por números, porque crearía un error de ejecución.

```
var #1 = 1; //Error de ejecución
var 1_1 = 1; //Error de ejecución
```

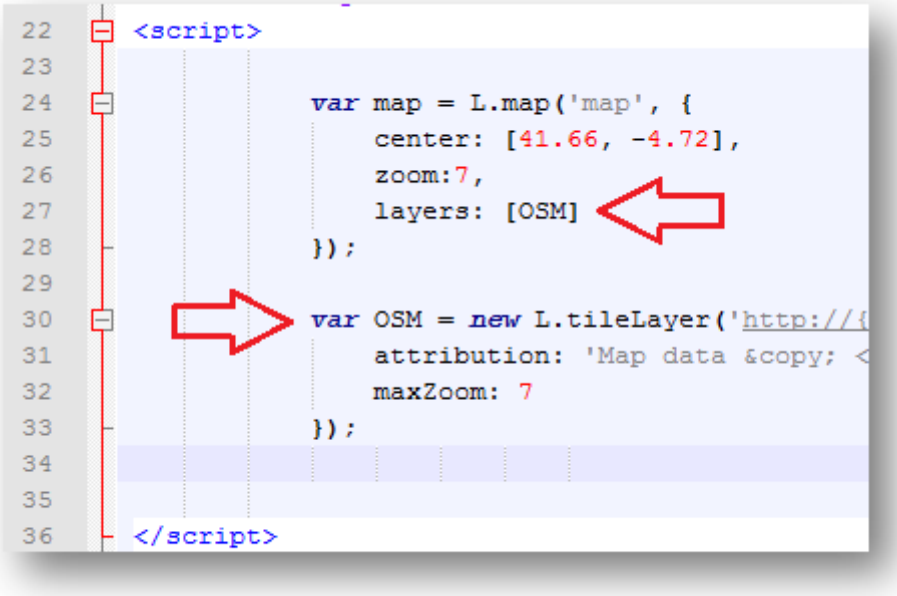
Tampoco podemos poner como nombre a una variable una palabra reservada.

Las variables son sensibles a mayúsculas y minúsculas:

`MiMapa` y `mimapa` son variables distintas.

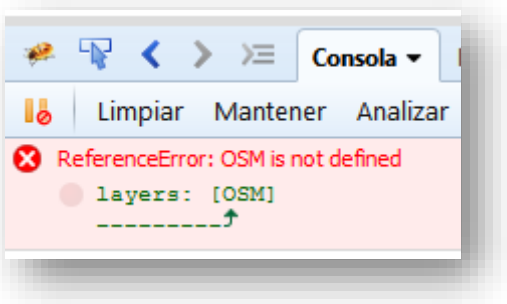
Orden en la creación de variables

JavaScript es un lenguaje de procedimientos, que va leyendo el código de arriba hacia abajo, por lo tanto no podemos utilizar una variable que aún no haya sido creada. Aunque se defina más adelantes. En este ejemplo vemos cómo NO hay que trabajar en JavaScript:



Este script no mostrará nada, ya que estamos intentando utilizar la capa de OSM en el mapa, pero el objeto OSM aun no ha sido creado, lo creamos en la línea 30.

El depurador de errores nos ayuda informándonos de que OSM no se ha definido:



Nota
Por el momento no te preocupes por la sintaxis utilizada ni por la depuración de errores, ya que lo veremos más adelante.

Aquí vemos el script correcto, donde primero creamos la variable y a continuación la utilizamos:

```
<script>

        var OSM = new L.tileLayer('ht
            attribution: 'Map data &c
            maxZoom: 7
        });

        var map = new L.map('map', {
            center: [41.66, -4.72],
            zoom: 7,
            layers: [OSM]
        });


</script>
```

Declaraciones y expresiones

Para entender la sintaxis de JavaScript, es muy útil saber que tiene dos grandes categorías sintácticas: declaraciones y expresiones.

Las **Declaraciones** "hacen cosas". Un programa es una secuencia de declaraciones. Ejemplo de una declaración, que declara (crea) una variable `foo`:

```
var foo;
```

Las **expresiones** producen valores. Son el lado derecho de una asignación, argumentos de funciones, etc. Ejemplo de una expresión:

```
2 + 1
```

Funciones útiles

Script con fecha

Un script es un programa JavaScript insertado en una página HTML.

Delimitado por marcaras <script> con atributo type=”text/javascript”.

document.write("Texto") inserta un texto en la página Web:

```
<!DOCTYPE html>
<html>
 <body>
<script type="text/javascript">
                document.write("Hoy es: " + new Date ());
</script>
```

```
</body>
</html>
```

Script con expresión

alert("Texto") muestra un mensaje al usuario en una ventana emergente:

```
<!DOCTYPE html>
<html>
 <body>
<script type="text/javascript">
                document.write("Hoy es: " + new Date ());
                alert ("Hola Mundo");
</script>

</body>
</html>
```

Otras funciones:

confirm () presenta un mensaje y pide confirmación o rechazo.

prompt (msj) presenta un mensaje y pide un dato de entrada.

Funciones

Una manera de definir una función es a través de una declaración de la función:

```
    function add (param1, param2) {
         return param1 + param2;
     }
```

El código anterior define una función **add** que tiene dos parámetros param1 y param2 y devuelve la suma de ambos parámetros. Así es como se llama a esa función:

```
> add(6, 1)
7

> add('a', 'b')
'ab'
```

Otra manera de definir add() es a través de una expresión de función:

```
var add = function (param1, param2) {
        return param1 + param2;
};
```

Esta función tiene cuatro partes.

1. Declara la función usando var, y luego asígnale un nombre.
2. Luego debes usar el comando function para decirle al computador que estás haciendo una función.
3. El fragmento entre paréntesis es llamado el **parámetro**. Piensa que es una palabra de reemplazo a la que le asignamos un valor específico cuando llamamos a la función.
4. Luego escribe tu código reusable entre { }. Cada línea de código en este bloque deber terminar con un ;

Condicionales y bucles

Los principales declaraciones en JavaScript para establecer condiciones son:

if, switch, for y while.

- IF

```
<script>

	var mivariable = 9;
	if(mivariable < 10) {
	alert("El valor es menor que 10");
	}

</script>
```

La declaración ***if*** controla la expresión entre paréntesis y procesa las instrucciones entre llaves si la condición es verdadera. En el código del listado anterior asignamos el valor 9 a **mivariable.** Como este valor es inferior al que indica la condición, nos mostrará el mensaje en la pantalla.

- SWITCH

La declaración ***switch*** se utiliza para comprobar múltiples condiciones.

```
<script>
	var mivariable = 3;
	switch(mivariable) {
		case 5:
			alert("El valor es cinco");
			break;
		case 8:
			alert("El valor es ocho");
			break;
		case 10:
```

```
                alert("El valor es diez");
                break;
        default:
                alert("El valor es " + mivariable);
    }
</script>
```

La declaración *switch* evalúa una expresión, compara el resultado con la opción **case** que están dentro de las llaves y devuelve el resultado correspondiente. En el ejemplo anterior creamos una variable a la que asignamos valor 3. A continuación se pasa a analizar las diferentes opciones. El primer valor es 5, como no es coincidente con el valor de **mivariable** pasa al segundo **case.** De esta forma continúa el proceso hasta que se cumpla alguna condición. En el ejemplo que hemos puesto, como no se cumple ninguna de las condiciones llega la final del proceso y muestra el valor por defecto.

- FOR

La declaración ***for*** ejecutará el código entre llaves mientras la condición indicada en el segundo parámetro sea verdadera. Utiliza la sintaxis **for (inicialización; condición, incremento).**

```
<script>
    var mivariable = 9;
    for(var f = 0; f < mivariable; f++) {
        alert("El valor actual es " + f);
    }
</script>
```

El primer valor de la condición for es f=0. Este será el valor de inicialización.

La condición que se marca es que ese valor sea menor que el indicado en **mivariable.**

En caso de que la condición sea verdadera el código entre llaves es ejecutado. Al finalizar el intérprete de JavaScript ejecuta el tercer parámetro de **for** (es decir f++, con el que se incrementará en una unidad el valor de f) y comprueba la condición nuevamente. Si la condición es verdadera, las instrucciones son ejecutadas una vez más. El bucle continúa hasta que la condición es falsa.

Ejercicio nº8. Inserta el código JavaScript anterior en un HTML. Ejecútalo y describe en la hoja de resultados lo que sucede.

- WHILE

La declaración while solo requiere la declaración de la condición entre paréntesis y el código a ser ejecutado entre llaves. El bucle se ejecutará hasta que la condición sea falsa. Veamos un ejemplo:

```
<script>
      var mivariable = 9;
            do {
               mivariable++;
               }while (mivariable > 100);
            alert( "El valor de mivariable es " + mivariable);
</script>
```

En este código estamos indicando como condición que "mivariable" sea mayor de 100. Como el valor asignado inicialmente a la variable fue 9, está condición es falsa, pero como estamos usando la condición do while, el código dentro del bucle es ejecutado una vez antes de que la condición sea evaluada. La ventana emergente mostrará el valor 10 (9+1)

Depuración de JavaScript

Hasta hace unos años, la depuración de JavaScript en sitios web era horrible; escribir código en un editor de texto, añadir declaraciones de **alerta** para inspeccionar valores, y repetir hasta que las cosas funcionasen.

El editor de texto sigue siendo aún la principal herramienta para los desarrolladores de JavaScript, pero actualmente ya existen ayudas como el resaltado de sintaxis, el autocompletado de código y otras herramientas específicas de lenguaje.

Dependiendo de dónde provengan los desarrolladores, las opciones más populares son vi, UltraEdit, TextMate o gedit. También hay plugins para IDEs como Eclipse o distribuciones completas de Eclipse realizadas para el desarrollo de JavaScript, como Aptana Studio.

JavaScript es ahora al menos tan importante como HTML para crear páginas web, y los principales navegadores incluyen ya herramientas de depuración para desarrolladores.

Corrección de errores con Firefox

Para depurar un script con Firefox, solamente es necesario acceder a la opción **Consola de error** dentro del menú **Herramientas.**

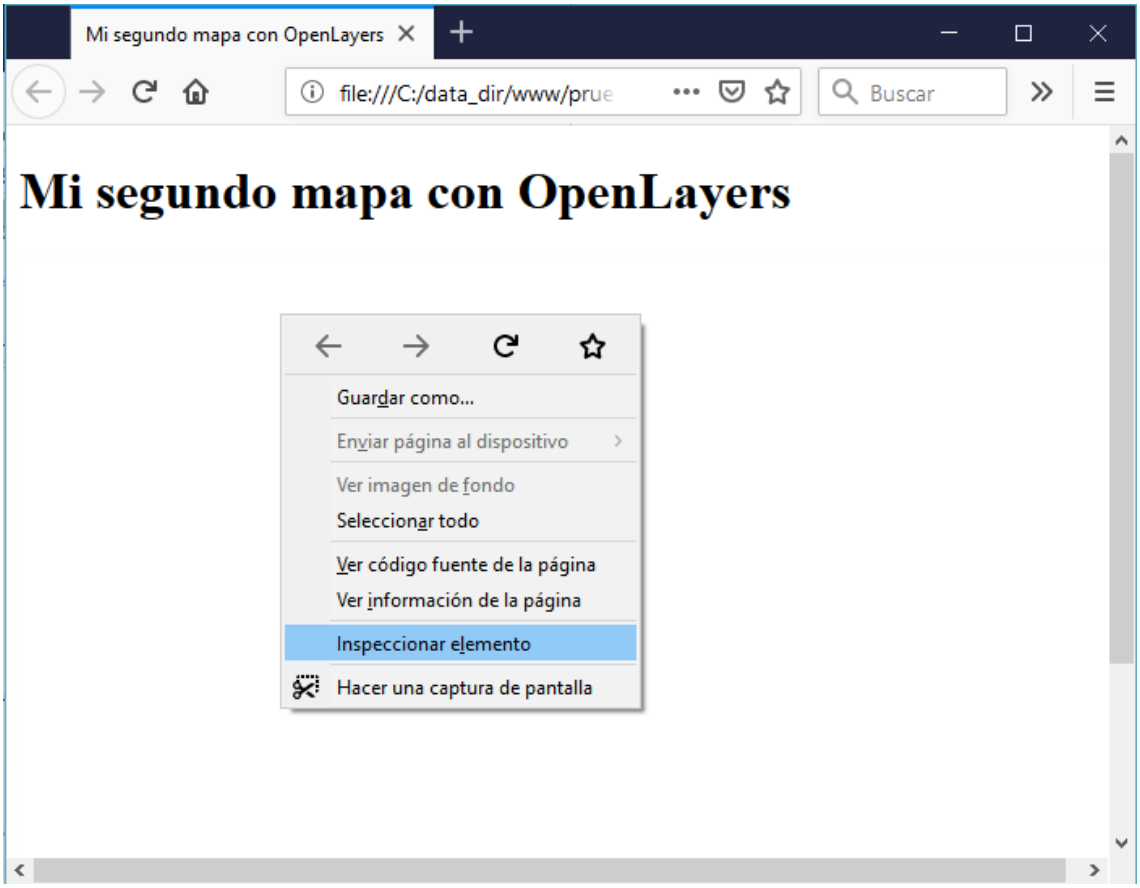
La consola de errores permite diferenciar los mensajes de información, los mensajes de aviso y los mensajes de error. Además, permite visualizar todos los errores de la página simultáneamente. Por cada error detectado se indica la posible solución mediante un mensaje en inglés y se muestra el trozo de código del script donde se ha producido el error. Además, pulsando sobre el enlace incluido se accede a la línea concreta del archivo concreto donde se ha producido el error.

Ejemplo práctico

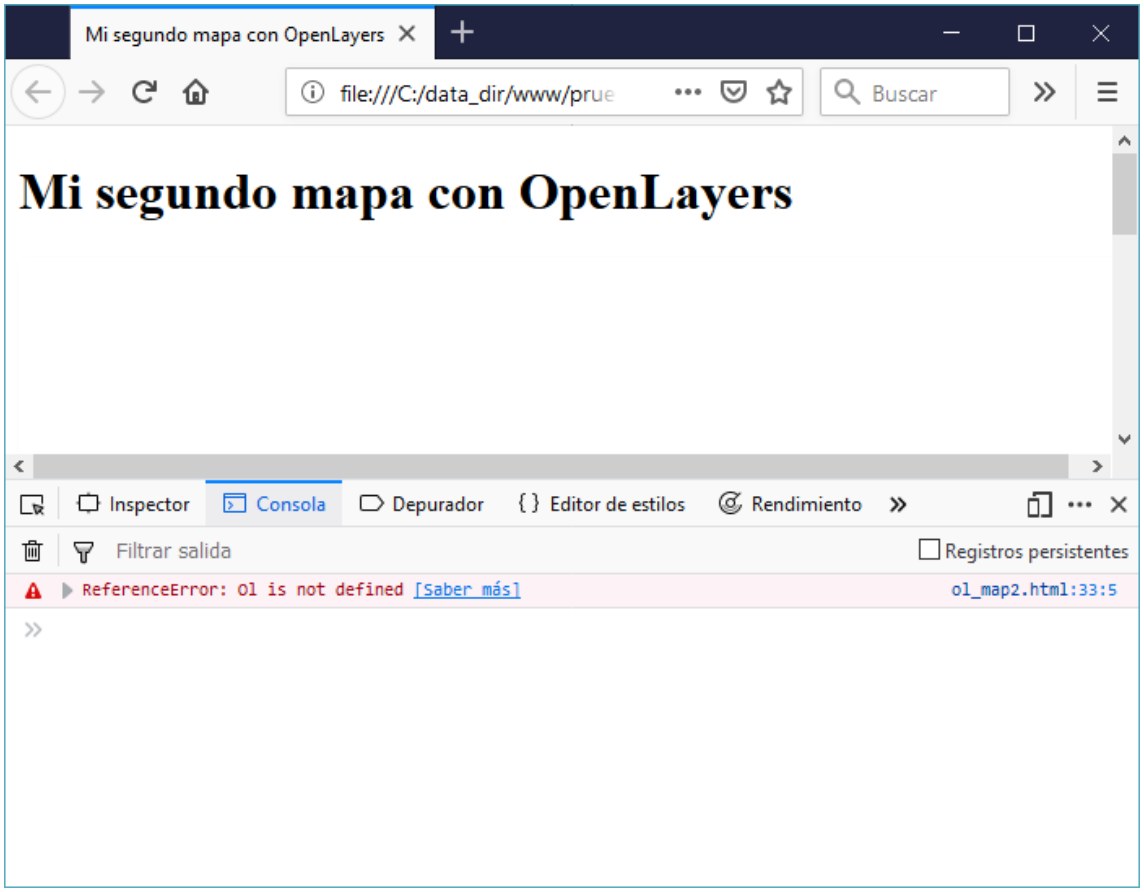
Hemos creado un HTML con la API de OpenLayers y el mapa no aparece, ¡qué horror!



Para acceder a las herramientas de desarrollo debemos hacer botón derecho **Inspeccionar elemento**:

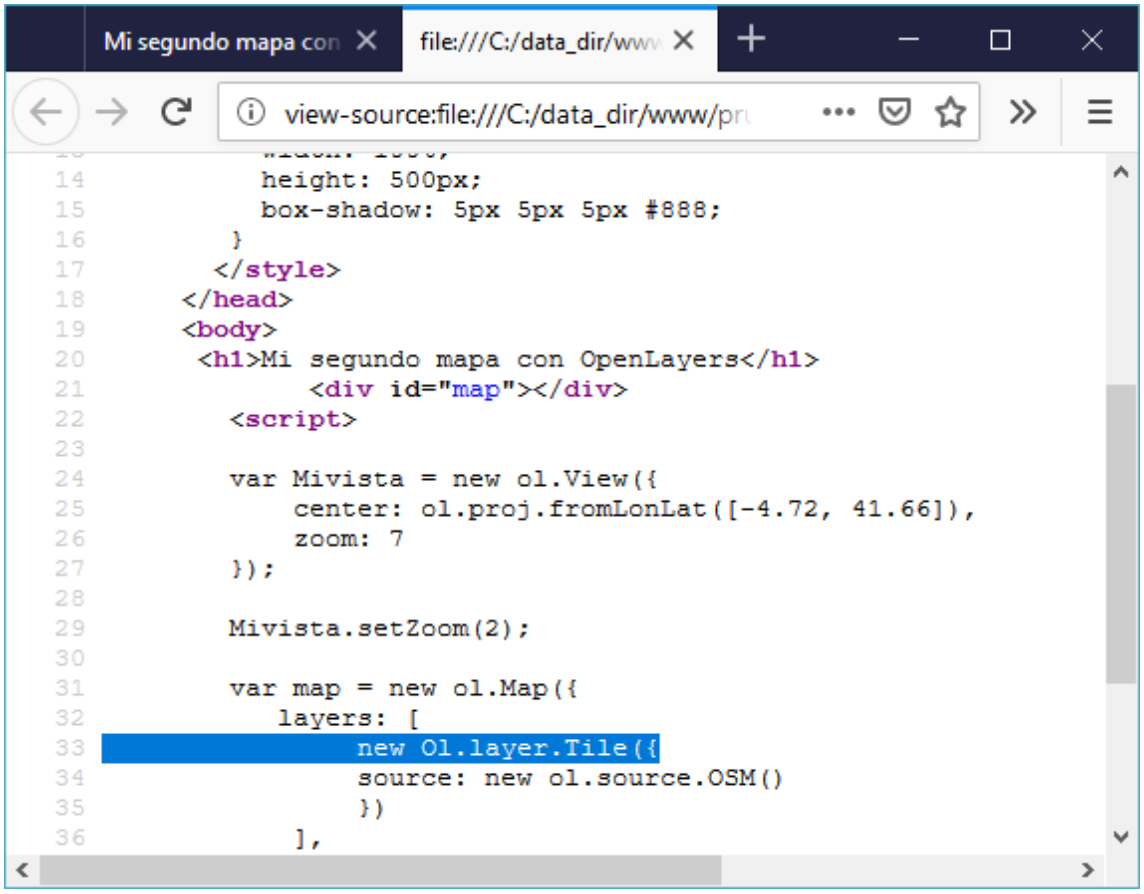


A continuación pinchamos sobre la pestaña Consola y vemos el error:



A la derecha se muestra la línea en la que se ha detectado un error, en este caso se trata de la línea 33, en la que hay un problema.

Si pinchamos ahí, se abrirá una nueva pestaña con la línea del código que debemos corregir:



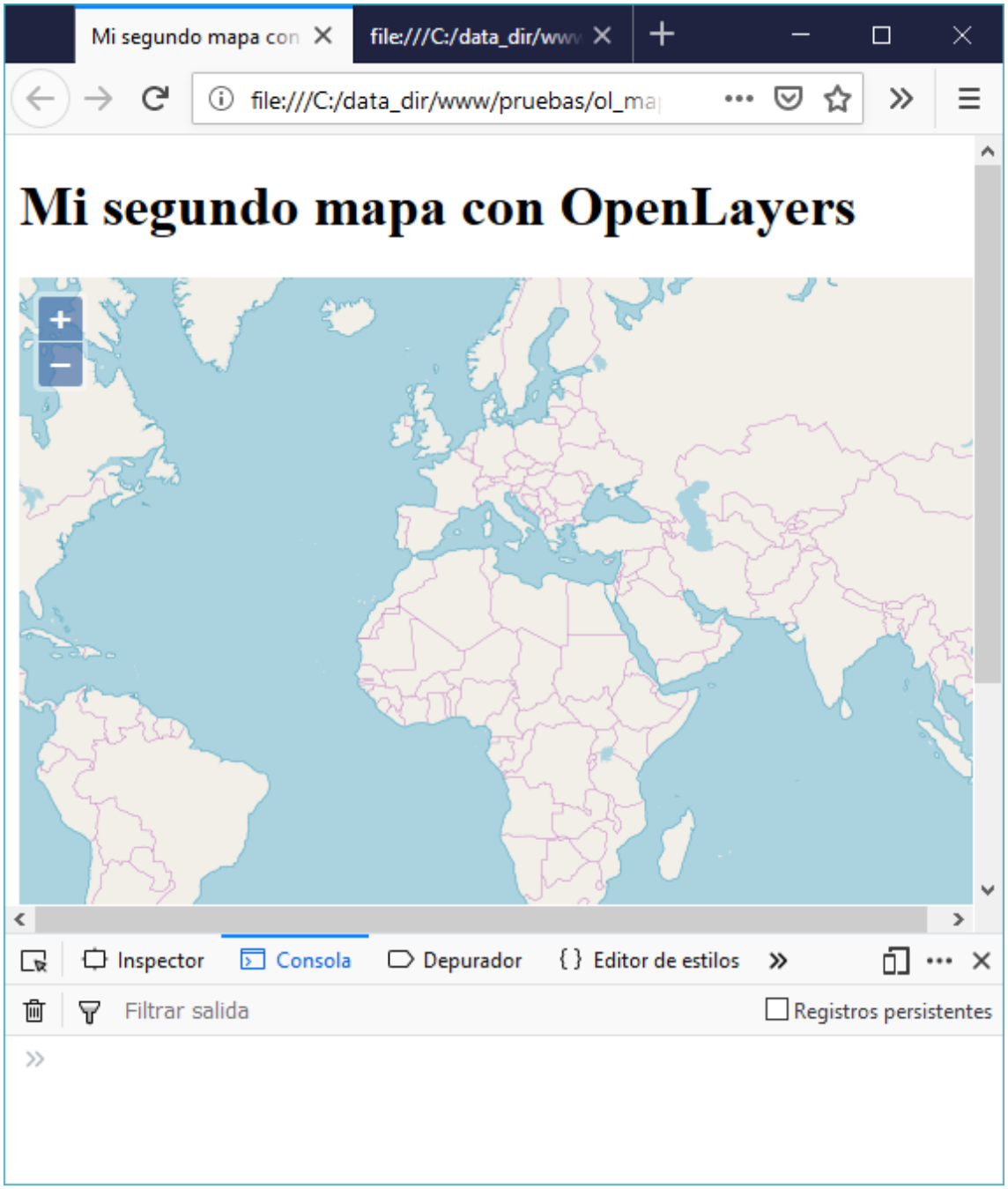
En este caso el problema es que hemos escrito

```
Ol.layer.Tile
```

Y debería poner

```
ol.layer.Tile
```

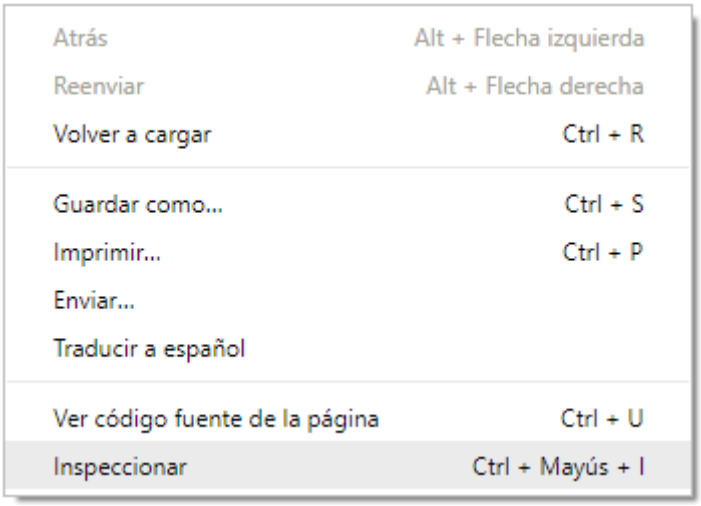
Corregimos el código **en nuestro editor de texto**, guardamos y actualizamos el navegador.



Aquí nuestro mapa sin errores.

Depuración de errores con Chrome

Al igual que con Firefox, una manera muy sencilla es hacer botón derecho con el ratón y seleccionar la opción **Inspeccionar.**

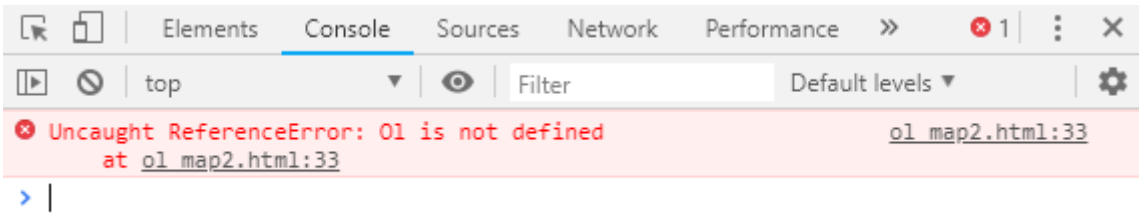


De este modo se abren las **Herramientas de desarrollo** de Google Chrome.

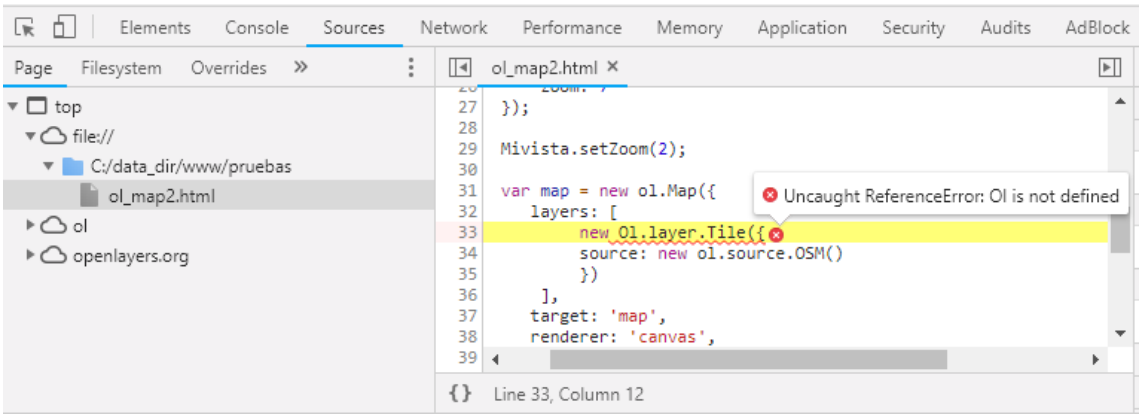
Las Herramientas de desarrollo pueden ser separadas de la ventana del navegador que estamos depurando, haciendo clic en el icono de la pantalla en la esquina inferior izquierda de las herramientas de desarrollo.

La primera pestaña, **Elements**, es útil para inspeccionar el estado actual de la DOM y para ver los estilos aplicados a todos los recursos de hojas de estilo. Ten en cuenta que tanto el DOM y los estilos se pueden modificar en esta ficha, para ver cómo podrían afectar a la apariencia de la página. Nótese, sin embargo, que estos cambios no se pueden conservar en la fuente de origen, por lo que son sólo temporales.

La pestaña **Console** mostrará los errores.



Al hacer clic sobre la línea en la que se encuentra el error, se mostrará el error en la pestaña **Sources**:



Corregimos el código **en nuestro editor de texto**, guardamos y actualizamos el navegador.

Ejercicio nº9. Nos han pasado una web pero su código tiene varios errores.

Utiliza un depurador para corregirlos y poder ver la web correctamente.

Este es el código:

```
<!DOCTYPE html>
<html

	<head>
		<title> Depuración de errores </title>
	<style>
	.p {
		font-family: Arial
		color: red;
		font-size 18px;
	}
</style>
	</head>

	<body>
		<!--Insertando una imagen -->
		<p Mapa de Leaflet</p>

		<img
src="https://upload.wikimedia.org/wikipedia/commons/thumb/5/53/Island_of_California.jpg/800px-Island_of_California.jpg"></img>
	       <div>
		<script type="text/javascript">
			var #x = Un mapa poco interactivo /creamos una variable
denominada x

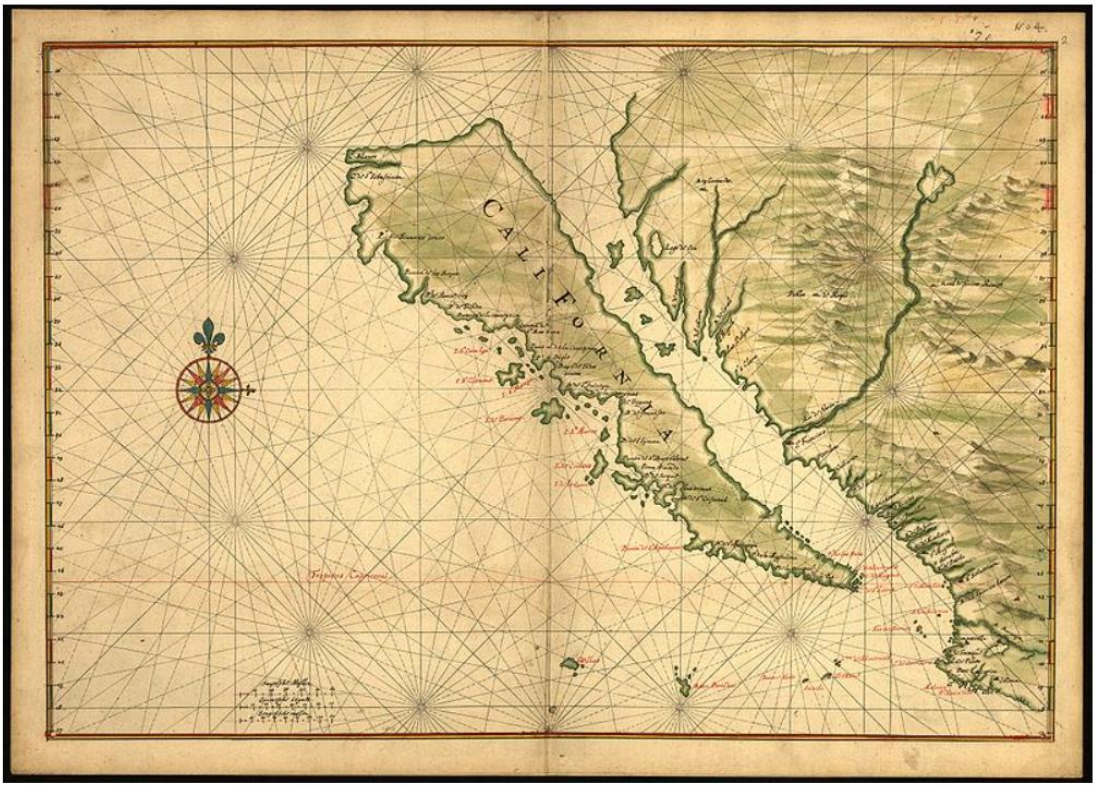
			document.write(y);

		</script>
		</div>
	</body>
</html>
```

Sube a la plataforma de formación el archivo HTML corregido y una captura de pantalla con la web.

Una vez corregido deberías ver lo siguiente:

Mapa de Leaflet



Un mapa poco interactivo